CS4052 Computación Paralela y Distribuida – 2026-II

Proyecto de Laboratorio – **Entrega final**

Caso 5: KNN en paralelo con MPI (mpi4py)

Clasificación de dígitos (*load_digits*)

Profesor: José Fiestas – Grupo 7

Integrante	**Participación**
Barzola Estrella, Johar Jared	100 %
Huaman Casma, Leonardo Fabian	100 %
Izaguirre Zavaleta, Luis Fernando	100 %

(100 % indica que el integrante contribuyó en forma equitativa al resto)

1. Introducción

KNN (*k-nearest neighbors*) es un algoritmo de clasificación supervisada: para clasificar un punto nuevo calcula su distancia a todos los puntos de entrenamiento, toma los k más cercanos y le asigna la etiqueta que más se repite entre ellos. En este proyecto paralelizamos con mpi4py [3] el código secuencial `knn_digits_sec.py` [2], que clasifica las imágenes de dígitos de *load_digits* [4]: 1797 imágenes de 8×8 píxeles ($d = 64$), divididas en $n_{tr} = 1437$ de entrenamiento y $n_{te} = 360$ de prueba, con $k = 3$. El código base da una precisión (*accuracy*) de 0,9833.

El trabajo más costoso es calcular las $n_{tr} \cdot n_{te} = 517\,320$ distancias, que son independientes entre sí. Siguiendo el DAG del enunciado [1] (Figura 1), el proceso 0 (maestro) envía la data de prueba a todos (`comm.bcast`), reparte la data de entrenamiento en bloques de n_{tr}/p (`comm.scatter`), cada proceso calcula sus k vecinos locales para cada punto de prueba y el maestro los recolecta (`comm.gather`), elige los k más cercanos de todos y vota.

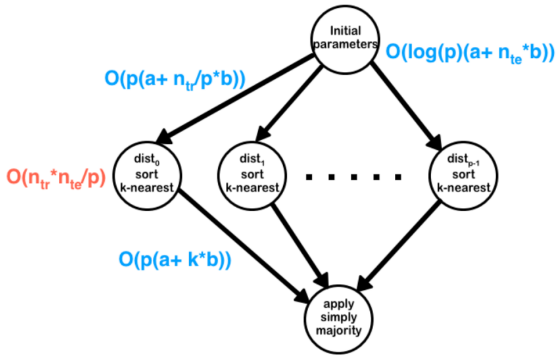


Figura 1: DAG del KNN en paralelo. En azul, los costos de comunicación; en rojo, el costo de cómputo de cada proceso. Los costos de scatter y gather corresponden a una topología lineal; MPI usa árboles (Sección 2.4). Fuente: enunciado del proyecto [1].

En la entrega parcial presentamos el PRAM y las métricas. Para esta entrega final escribimos el código en tres versiones beta y una mejora con reducción en árbol, las validamos contra el código secuencial, medimos los tiempos de ejecución, cómputo y comunicación para $p \in \{1, 2, 4, 6, 8\}$ y cuatro tamaños del problema (de $n = 449$ a $n = 28\,752$), normalizamos la expresión teórica para compararla con las mediciones y analizamos el *speedup*, los FLOP/s y la escalabilidad. Los resultados principales son:

- Las tres versiones y la mejora con reducción dan exactamente las mismas predicciones que el código base para $p = 1$ a 8 (precisión 0,9833).

- La versión final (beta 3) es 38 veces más rápida que el código base en un solo proceso, y con $p = 8$ llega a $S = 7{,}2$ con $n = 1797$ y $S = 9{,}2$ con $n = 28\,752$.
- El modelo teórico normalizado reproduce los tiempos medidos con $R^2 = 0{,}998$ y un error relativo medio de 8 %.
- La cantidad óptima de procesos crece con el tamaño del problema: $p_{opt} \approx 7$ para $n = 449$ y $p_{opt} \approx 18$ para $n = 1797$.
- El algoritmo escala si $n = \Theta(k\,p^2)$ (isoeficiencia); los resultados con $p \leq 8$ son consistentes con esa condición.
- La reducción en árbol (`comm.reduce`) acorta los pasos 5 a 7 hasta un 33 % con $p = 8$, pero el tiempo total casi no cambia: el ahorro es pequeño.

2. Método

2.1. Algoritmo secuencial y tamaño del problema

Llamamos n al número total de imágenes. El código base divide los datos en 80/20, así que $n_{tr} = 0{,}8\,n$ y $n_{te} = 0{,}2\,n$: basta un solo n para describir el problema, y las dos partes crecen juntas.

Por cada punto de prueba, el código base calcula n_{tr} distancias euclidianas de $O(d)$ operaciones, las ordena con `np.argsort` ($O(n_{tr} \log n_{tr})$) y vota entre los k vecinos con `Counter` ($O(k)$). Entonces

$$T_s(n) = O\big(n_{te}\,(n_{tr}\,d + n_{tr} \log n_{tr} + k)\big) = O(n^2 \log n),$$

porque d y k son constantes. Con los tamaños que usamos, $d = 64 > \log_2 n_{tr}$ ($\approx 10{,}5$ con los datos originales y a lo más 16,5), así que esperábamos que el cálculo de distancias, $O(n_{tr}\,n_{te}\,d) = O(n^2)$, pesara más que el ordenamiento. Ese es el costo de cómputo del DAG, $O(n_{tr} \cdot n_{te})$, y también el del mejor algoritmo secuencial, que elige los k menores sin ordenar todo (por ejemplo, con `np.argpartition`). (En la Sección 3.3 vemos que en la beta 3 el término del ordenamiento pesa más de lo que supusimos.)

2.2. Diseño con el método de Foster

Seguimos los cuatro pasos del método de Foster tal como los ve el curso (U1.2 y la PD01 [5], [6]).

Particionamiento. Usamos descomposición del dominio sobre la data de mayor tamaño, la de entrenamiento. La tarea primitiva es la distancia entre un punto de prueba y uno de entrenamiento: $n_{tr} \cdot n_{te} = 517\,320$ tareas con los datos originales, todas de costo $O(d)$ y muchas más que procesos (Figura 2).

Comunicación. Cada tarea necesita sus dos puntos y entrega su distancia a la selección de los k vecinos de su punto de prueba: las tareas de una misma fila se combinan en una reducción, con del orden de $n_{tr} \cdot n_{te}$ canales. Esta comunicación es *overhead*: no existe en el algoritmo secuencial.

Aglomeración. Agrupamos en una sola tarea todas las distancias de un bloque de n_{tr}/p puntos de entrenamiento contra los n_{te} puntos de prueba, y hacemos que cada bloque devuelva solo sus k mejores candidatos por punto de prueba en lugar de todas sus distancias. Así quedan del orden de $3p$ canales: un broadcast, un scatter y un gather por bloque. Elegir solo k candidatos por bloque no cambia el resultado: si un punto es uno de los k vecinos más cercanos entre todos los datos de entrenamiento, también lo es dentro de su bloque (si no, el bloque tendría k puntos más cercanos y el punto no podría ser vecino global). **Esta aglomeración es la que convierte el PRAM ideal de la Sección 2.3 en el algoritmo distribuido del DAG** (Sección 2.4): el paralelismo baja de $n_{tr} \cdot n_{te}$ tareas a p tareas y la comunicación baja de una distancia por tarea a tres colectivas.

Mapeo. Un bloque por proceso, estático. Como n_{tr} puede no ser múltiplo de p (por ejemplo, 1437), unos procesos reciben una fila más que otros; en el análisis asumimos bloques balanceados de tamaño n_{tr}/p y medimos el desbalance real (Sección 3.7).

El enunciado también permite repartir los datos de prueba [1]: con un scatter de la prueba y un broadcast del entrenamiento no hace falta la mezcla del paso 6 en el maestro, pero cada proceso recibe

todo el entrenamiento, que es el conjunto más grande. Seguimos el DAG, que reparte el de mayor tamaño.

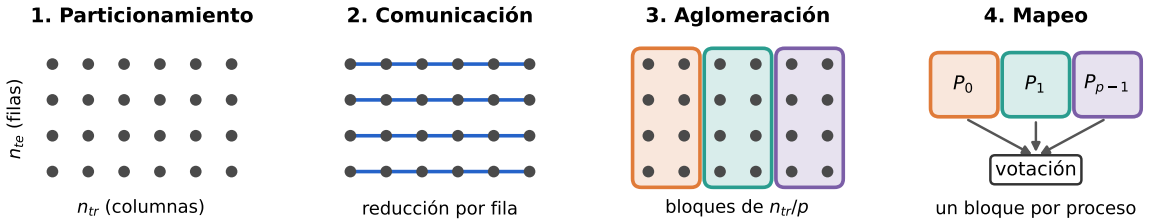


Figura 2: Método de Foster aplicado al KNN. Cada punto es una tarea: la distancia entre un punto de prueba (fila) y uno de entrenamiento (columna). Fuente: elaboración propia.

2.3. PRAM: trabajo y span

El Algoritmo 1 es el KNN en el modelo PRAM, sin ninguna referencia a p: se supone un procesador por tarea. Así el trabajo y el span se leen directamente del pseudocódigo. El modelo es CREW: todos los procesadores leen los mismos datos y cada uno escribe en su propia posición. Para que las dos reducciones en árbol (pasos 3 y 6) queden exactas suponemos, como en el ejemplo de la suma de n números del curso [5], que n_{tr} y n_{te} son potencias de 2; si no lo son, se completan con posiciones vacías ($C[i,j] = \emptyset$ y $A[i] = 0$), que no cambian el resultado y a lo más duplican cada tamaño. En el paso 3, si la unión tiene menos de k índices, se conservan todos.

Algorithm 1: KNN en PRAM$(X_{tr}, y_{tr}, X_{te}, y_{te}, k)$ — sin p

Entrada: datos de entrenamiento X_{tr} ($n_{tr} \times d$) con etiquetas y_{tr}; datos de prueba X_{te} ($n_{te} \times d$) con etiquetas y_{te}; k vecinos

Salida: etiquetas predichas $\hat{y}[1, \ldots, n_{te}]$ y precisión (*accuracy*)

1 **for** $(i,j) \in \{1,\ldots,n_{te}\} \times \{1,\ldots,n_{tr}\}$ **pardo**
2 $D[i,j] \leftarrow \sqrt{\sum_{l=1}^{d} (X_{te}[i,l] - X_{tr}[j,l])^2}$ // paso 1
3 $C[i,j] \leftarrow \{\, j \,\}$ // paso 2
4 **for** $h \leftarrow 1$ **to** $\log_2 n_{tr}$ **do**
5 **for** $(i,j) \in \{1,\ldots,n_{te}\} \times \{1,\ldots,n_{tr}/2^h\}$ **pardo**
6 $C[i,j] \leftarrow$ los k índices de menor $D[i,\cdot]$ en $C[i,2j-1] \cup C[i,2j]$ // paso 3
7 **for** $i \leftarrow 1$ **to** n_{te} **pardo**
8 $\hat{y}[i] \leftarrow$ etiqueta más frecuente entre las $y_{tr}[j]$ con $j \in C[i,1]$ // paso 4
9 $A[i] \leftarrow 1$ si $\hat{y}[i] = y_{te}[i]$, si no 0 // paso 5
10 **for** $h \leftarrow 1$ **to** $\log_2 n_{te}$ **do**
11 **for** $i \leftarrow 1$ **to** $n_{te}/2^h$ **pardo**
12 $A[i] \leftarrow A[2i-1] + A[2i]$ // paso 6
13 *accuracy* $\leftarrow A[1] \,/\, n_{te}$ // paso 7
14 **return** $\hat{y}$, *accuracy*

Trabajo. Contamos las operaciones paso por paso, como en el ejemplo de la suma de n números del curso [5]. En los pasos 1 y 2 se ejecutan $n_{tr}n_{te}$ distancias de $3d$ operaciones y $n_{tr}n_{te}$ inicializaciones; en el paso 3, $\sum_{h=1}^{\log_2 n_{tr}} n_{te}\, n_{tr}/2^h$ mezclas de $O(k)$; en los pasos 4 y 5, n_{te} votaciones entre k y n_{te} comparaciones; en el paso 6, $\sum_{h=1}^{\log_2 n_{te}} n_{te}/2^h$ sumas; y en el paso 7, una división. Como $\sum_{h\geq 1} n_{tr}/2^h < n_{tr}$ y d y k son constantes,

$$W(n) = O(n_{tr}\, n_{te}\, d + k\, n_{tr}\, n_{te} + n_{te}\, k + n_{te}) = O(n_{tr}\, n_{te}) = O(n^2),$$

que es el costo del mejor algoritmo secuencial (Sección 2.1): el PRAM no ordena, solo mezcla listas de k.

Span. Los *pardo* cuentan un paso y los *for* secuenciales multiplican [5]: los pasos 1 y 2 cuestan un paso, el bucle del paso 3 cuesta $\log_2 n_{tr}$, los pasos 4 y 5 uno, el bucle del paso 6 cuesta $\log_2 n_{te}$ y el paso 7 uno. Dentro de cada paso, un solo procesador hace la suma de los d términos de una distancia, la votación entre k etiquetas o la mezcla de dos listas de k, en $O(d)$, $O(k)$ y $O(k)$ operaciones, o sea $O(1)$ porque $d = 64$ y $k = 3$ son constantes. Entonces

$$T_\infty(n) = O(\log n_{tr} + \log n_{te} + 3) = O(\log n).$$

Brent. Con W y T_∞, el tiempo con p procesadores cumple [5]

$$T_p(n,p) = O\left(\frac{W(n)}{p} + T_\infty(n)\right) = O\left(\frac{n^2}{p} + \log n\right),$$

es decir, el algoritmo es casi perfectamente paralelo: el span es logarítmico y el trabajo se reparte. El costo es $C_p(n,p) = p\,T_p(n,p) = O(W(n) + p\,T_\infty(n)) = O(n^2 + p\log n)$, así que el algoritmo es de costo óptimo ($C_p = \Theta(W)$, el costo del mejor secuencial) mientras $p = O(n^2/\log n)$ [5]: con los tamaños y los $p \le 8$ que usamos, eso se cumple de sobra. Brent supone que no hay costo de comunicación [5]; la Sección 2.4 agrega ese costo.

2.4. Algoritmo distribuido (el del DAG)

El Algoritmo 2 es la versión con p procesos que sigue el DAG del enunciado, y es la aglomeración descrita en la Sección 2.2. Las líneas 1, 2 y 9 son `comm.bcast`, `comm.scatter` y `comm.gather`.

Algorithm 2: KNN distribuido con Broadcast/Scatter/Gather$(X_{tr}, y_{tr}, X_{te}, y_{te}, k, p)$

Entrada: datos de entrenamiento X_{tr} ($n_{tr} \times d$) con etiquetas y_{tr}; datos de prueba X_{te} ($n_{te} \times d$) con etiquetas y_{te}; k vecinos; p procesos, q el ID del proceso ($0 \le q \le p-1$)

Salida: etiquetas predichas $\hat{y}[1,\dots,n_{te}]$ y precisión (*accuracy*) en el proceso 0

1 Broadcast$(X_{te}, k, root = 0)$ `// paso 1`
2 Scatter$((X_{tr}, y_{tr}), (X_q, y_q), root = 0)$ `// paso 2`
3 **for** $q \leftarrow 0$ **to** $p-1$ **pardo**
4 **for** $i = 1$ **to** n_{te} **do**
5 **for** $j = 1$ **to** n_{tr}/p **do**
6 $D[j] \leftarrow \sqrt{\sum_{l=1}^{d} (X_{te}[i,l] - X_q[j,l])^2}$ `// paso 3`
7 $I \leftarrow$ índices de los k menores valores de D (ordenar D) `// paso 4`
8 $C_q[i] \leftarrow \{(D[j], y_q[j]) : j \in I\}$
9 Gather$(C_q, C, root = 0)$ `// paso 5`
10 **if** $q = 0$ **then**
11 **for** $i = 1$ **to** n_{te} **do** `// paso 6`
12 $V \leftarrow$ los k pares de menor distancia en $C[i]$
13 $\hat{y}[i] \leftarrow$ etiqueta más frecuente en V
14 $accuracy \leftarrow$ (cantidad de i con $\hat{y}[i] = y_{te}[i]$) / n_{te} `// paso 7`
15 **return** $\hat{y}, accuracy$

Usamos el modelo de comunicación del curso [5], con $a \equiv \alpha$ (latencia) y $b \equiv \beta$ (tiempo por dato) como en el DAG. El DAG del enunciado usa una topología lineal, en la que el scatter y el gather de m datos cuestan $O(a\,p + b\,m)$: pagan p veces la latencia, y es una aproximación no optimizada. MPI organiza las colectivas en árbol. Así, el broadcast cuesta $O(\log p\,(a + m\,b))$, y el scatter y el gather $O(a\log p + b\,m)$: la latencia se paga en $\log p$ niveles, pero el maestro igual envía o recibe los m datos [5]. Así nos lo aclaró el profesor [10], y con esos costos, escritos con un solo n, queda la Tabla 1. En

el DAG el gather cuesta $O(p(a + k\,b))$, que es el costo para un punto de prueba; nosotros hacemos un solo Gather con los candidatos de los n_{te} puntos (para no repetir la latencia n_{te} veces), así que el maestro recibe $m = p\,n_{te}\,k$ pares. T_{comp_2} no aparece en el DAG, pero es el trabajo de su último nodo (votación), que hace el maestro.

Paso	**Líneas**	**Qué se ejecuta**	**Costo**
1	1	Broadcast de los $0{,}2n$ datos de prueba	$T_{comm_1} = O(\log(p)(a + n\,b))$
2	2	Scatter de $0{,}8n/p$ datos a cada proceso	$T_{comm_2} = O(a \log p + n\,b)$
3 y 4	3–8	$0{,}16\,n^2/p$ distancias y ordenamiento	$T_{comp_1} = O\big(\frac{n^2}{p}(d + \log \frac{n}{p})\big) \approx O\big(\frac{n^2}{p}\big)$
5	9	Gather de k candidatos por punto de prueba	$T_{comm_3} = O(a \log p + n\,k\,p\,b)$
6 y 7	10–15	Selección de k entre $p \cdot k$, votación y precisión	$T_{comp_2} = O(n\,p\,k)$

Tabla 1: Costo de cada paso del Algoritmo 2, con $n_{tr} = 0{,}8n$ y $n_{te} = 0{,}2n$. La aproximación de T_{comp_1} vale mientras $\log_2(n/p) < d$.

Sumando los pasos, y con $S = T_s/T_p$ y $E = S/p$ [5]:

$$T_p(n,p) = O\Big(\frac{n^2}{p} + n\,k\,p + a \log p + b\,\big(n + n \log p + n\,k\,p\big)\Big)$$

$$E(n,p) = O\left(\frac{1}{1 + \dfrac{k\,p^2}{n} + \dfrac{p \log p}{n} + \dfrac{p}{n} + \dfrac{p \log p}{n^2}}\right) = O\left(\frac{1}{1 + \dfrac{k\,p^2}{n}}\right)$$

En el último paso quitamos los términos menores: $p/n \le k\,p^2/n$ y $p \log p/n \le k\,p^2/n$ (porque $1 \le k\,p$ y $\log_2 p \le k\,p$ con $k \ge 1$), y $p \log p/n^2 \le k\,p^2/n$. Queda un solo término de *overhead*, $k\,p^2/n$, que viene del gather de los candidatos y de la mezcla en el maestro: la eficiencia se mantiene alta mientras $n \gg k\,p^2$.

2.5. Código en mpi4py

El Listado 1 muestra la parte central de la versión final. Las colectivas van en minúscula (objetos de Python), con `root=0`. Como n_{tr} no es múltiplo de p, el maestro reparte con `np.array_split`, que deja bloques que difieren en a lo más una fila. Cada candidato lleva su índice global para desempatar distancias iguales en el paso 6.

Listado 1: Parte central de `knn_mpi_beta3.py` simplificada: sin las barreras ni las marcas de tiempo intermedias de la medición (los pasos corresponden al Algoritmo 2).

```
t0 = MPI.Wtime()
X_te, k_l = comm.bcast((X_test, k), root=0) # 1. Broadcast de la data de prueba
if rank == 0: # 2. Scatter de la data de entrenamiento
    idx = np.arange(len(X_train))
    bloques = list(zip(np.array_split(X_train, size),
                       np.array_split(y_train, size),
                       np.array_split(idx, size)))
else:
    bloques = None
X_q, y_q, idx_q = comm.scatter(bloques, root=0)
C_q = k_vecinos_locales(X_te, X_q, y_q, idx_q, k_l) # 3-4. k vecinos locales
C = comm.gather(C_q, root=0) # 5. Gather de los candidatos
# beta 3b: C = comm.reduce(C_q, op=mezcla, root=0) # 5-6. reduccion en arbol
if rank == 0:
```

```
    y_pred = mezclar_y_votar(C, k_l) # 6. k globales y votacion
    # beta 3b: y_pred = votar(C[1]) # 6. solo la votacion
    acc = np.mean(y_pred == y_test) # 7. Precision
tf = MPI.Wtime()
```

2.6. Versiones beta y validación

Las tres versiones tienen la misma estructura (el DAG y el Algoritmo 2); lo que cambia es cómo se calcula el cómputo local:

- **Beta 1:** traducción directa del Algoritmo 2. Las distancias se calculan como en el código base, con un bucle de Python y `euclidean_distance` por cada par.
- **Beta 2:** para cada punto de prueba se calculan con NumPy todas las distancias al bloque local en una sola operación; la mezcla y la votación del maestro también se vectorizan. Con ella empezamos a medir por fases y a guardar las repeticiones en un archivo CSV; para la entrega final, las tres betas se miden con el mismo protocolo.
- **Beta 3 (final):** la matriz de distancias de un bloque de hasta 512 puntos de prueba se calcula con $\|x-y\|^2 = \|x\|^2 + \|y\|^2 - 2\,x \cdot y$, que es la misma distancia pero usa un producto de matrices. Se agrega el tamaño variable del problema y la versión secuencial del mismo cálculo (para T_s).
- **Beta 3b (mejora):** igual a la beta 3, pero el gather de los candidatos y la mezcla del maestro se reemplazan por `comm.reduce` con una operación que deja los k menores de dos listas; así la mezcla se hace en árbol, en $\log_2 p$ niveles (Sección 3.8).

Validación. Como el código lo escribió una herramienta de IA [9], lo validamos comparando las predicciones (`validar.py`, 49 casos). Con los datos originales, las betas 1, 2, 3 y 3b dan exactamente las mismas 360 predicciones que el código base para $p = 1$ a 8, con precisión 0,9833 (Tabla 2). Con $n/4$ y $4n$ ($p = 4$ y 8), las betas 3 y 3b dan las mismas predicciones que el código base y que la versión secuencial de la beta 3, y en la serie débil también la beta 3; con $16n$ ($p = 8$), las dos dan las mismas que la secuencial. También probamos bloques con menos de k puntos ($n_{tr} = 7$ con $p = 8$), que se completan con candidatos de distancia infinita. En 7 de los 360 puntos de prueba hay un empate de distancia en el tercer vecino (`pruebas_extra.py`); el desempate por índice global puede elegir otro vecino que `np.argsort`, pero las predicciones siguen siendo las mismas, como muestra la validación. El código, los scripts y los datos medidos van en el .zip adjunto (ver `README.md`).

Versión	Accuracy	$\mathbf{T_{ej}(p{=}1)}$ (ms)	$\mathbf{T_{ej}(p{=}8)}$ (ms)	$\mathbf{T(1)/T(8)}$
beta 1	0,9833	1034,1	164,9	6,27
beta 2	0,9833	53,9	10,3	5,24
beta 3	0,9833	27,0	3,7	7,21
beta 3b	0,9833	26,6	3,8	7,05

Tabla 2: Versiones beta con los datos originales ($n = 1797$), mediana de 4 repeticiones. T_{ej} es el tiempo de la región paralelizable, definido en la Sección 2.7. En todas, las predicciones son iguales a las del código base para $p = 1$ a 8. El código base tarda 1024 ms.

2.7. Diseño experimental

Entorno. El clúster Khipu no estuvo disponible por un problema de *firewall* de la universidad; el profesor indicó que en ese caso estaba justificado medir en una computadora personal [10]. Medimos en una Apple M1 Pro con 10 núcleos (8 de rendimiento y 2 de eficiencia) y 16 GB de RAM, con macOS 27.2.1, Python 3.11.6, NumPy 2.4.6, scikit-learn 1.9.1, mpi4py 4.1.2 y Open MPI 5.0.11, todo en una sola sesión de 33 min. Usamos $p \leq 8$ para que haya a lo más un proceso MPI por núcleo de rendimiento [7]; macOS no permite fijar cada proceso a un núcleo, así que el sistema operativo decide dónde corre cada uno. Cada proceso usa un solo hilo (`OMP_NUM_THREADS=1`, `VECLIB_MAXIMUM_THREADS=1`). Cerramos OneDrive, Docker y las aplicaciones que no hacían falta. La sesión fue a batería (sin modo de bajo consumo) y con Claude Code abierto, que lanzó el barrido; su uso de CPU queda dentro de

la carga externa. Antes de cada corrida registramos la carga externa, es decir, los núcleos que usaban otros programas (mediana 0,9; máximo 3,5), y en cada corrida el menor cociente entre tiempo de CPU y tiempo real de los procesos (mediana 0,996). Ese cociente fue menor que 0,9 en 18 de 490 corridas, 15 de ellas con $p = 8$: ahí otro programa le quitó CPU a algún proceso.

Tamaños del problema. Además de los datos originales (n_0), usamos un subconjunto ($n/4$: las primeras filas de cada parte) y dos tamaños mayores ($4n$ y $16n$); los nombres $n/4$, $4n$ y $16n$ son siempre relativos a n_0. Para crecer hicimos réplicas de las imágenes con ruido gaussiano ($\sigma = 1$ píxel, semilla fija) después del *split* del código base, sin mezclar entrenamiento con prueba; así se mantienen $d = 64$, la proporción 80/20 y una precisión con sentido (Tabla 3 y Figura 3). El profesor también usa datos sintéticos para escalar el KNN (`knn.cpp`, 20 000/5000 puntos [8]). Como la proporción 80/20 se mantiene, cada tamaño queda descrito por un solo n.

Series de escalabilidad. Además de las series con n fijo (escalabilidad fuerte), medimos dos series donde n crece con p: la **débil**, con $n \propto \sqrt{p}$, que deja constante el trabajo por proceso ($W/p = 0{,}16\, n^2/p$), y la de **isoeficiencia**, con $n \propto k\,p^2$, que es la condición de la Sección 3.6. Las réplicas admiten factores no enteros, así que n puede crecer como $\sqrt{p}$. Medimos además una serie de **control** con n_{te} fijo y $n_{tr} \propto p$ (dos n separados): ahí el trabajo por proceso también es constante, pero el largo de las listas que ordena cada proceso no cambia, así que sirve para aislar el efecto del hardware compartido (Sección 3.7).

Tamaño	$\mathbf{n_{tr}}$	$\mathbf{n_{te}}$	Pares	FLOP	Accuracy	$\mathbf{T_s}$ (ms)
$n/4$	359	90	32 310	$6{,}20 \cdot 10^6$	0,9778	1,21
n_0	1 437	360	517 320	$9{,}93 \cdot 10^7$	0,9833	26,81
$4n$	5 748	1 440	8 277 120	$1{,}59 \cdot 10^9$	0,9812	517,97
$16n$	22 992	5 760	132 433 920	$2{,}54 \cdot 10^{10}$	0,9807	9754,85

Tabla 3: Tamaños del problema. FLOP $= 3d \cdot n_{tr} \cdot n_{te}$; T_s es la versión secuencial de la beta 3 (mediana).

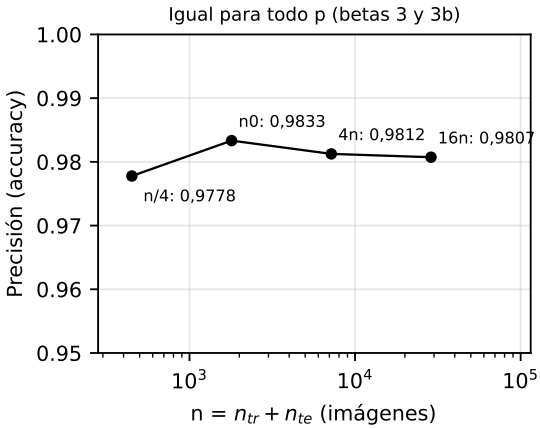


Figura 3: Precisión de cada tamaño del problema; es la misma para todo p y en las betas 3 y 3b. Fuente: elaboración propia.

Medición. Medimos con `MPI.Wtime()` en el maestro, desde antes del broadcast hasta después de calcular la precisión (la carga y la división de los datos quedan fuera, como en el código base). Cada `mpirun` mide una sola repetición de una configuración, y el script recorre toda la malla repetición por repetición (en las impares, en orden inverso). Así las repeticiones de una misma configuración quedan separadas en el tiempo, y una racha de carga no afecta siempre a la misma. Hicimos 4 repeticiones (2 en el tamaño mayor de isoeficiencia, cuya versión secuencial tarda 3 min) y reportamos la mediana; las barras de las figuras van de la repetición más rápida a la más lenta. Antes de medir, cada `mpirun` corre rondas de calentamiento que se descartan hasta juntar unos 0,25 s (entre 1 y 30 rondas; ninguna en los tamaños de más de $2 \cdot 10^8$ pares). Las configuraciones que se repiten en varias series (por ejemplo, n_0 con $p = 1$) se miden una sola vez, así que todas las series comparten la misma línea base.

Hicimos dos modos: en el modo A no hay barreras dentro de la región medida y de ahí sale T_{ej} (con el que calculamos S, E y FLOP/s); en el modo B ponemos `comm.Barrier()` antes de cada marca solo para separar las fases, porque sin barreras el gather incluiría la espera del proceso más lento. Los dos modos se miden en el mismo `mpirun`, en orden alternado. Definimos $T_{comm} = T_{bcast} + T_{scatter} + T_{gather}$ y $T_{comp} = T_{comp_1} + T_{comp_2}$, con T_{comp_1} el del proceso más lento. Como T_s usamos la versión secuencial del mismo cálculo de la beta 3, medida de la misma forma. No es el mejor algoritmo secuencial [8]: con selección de los k menores (`np.argpartition`) sería más rápido y S sería menor (Secciones 3.5 y 3.8); la usamos para que S compare el mismo cálculo repartido entre p procesos.

2.8. FLOP de la región paralelizable

La región paralelizable es el cálculo de distancias entre los puntos de prueba y los de entrenamiento. Según la fórmula de la distancia euclidiana, cada par necesita d restas, d productos, $d-1$ sumas y una raíz; contando todas las operaciones igual [8]:

$$\text{FLOP por par} = 3d = 192, \qquad \text{FLOP}_{total} = 3d\, n_{tr}\, n_{te} = 99\,325\,440 \ \text{ con los datos originales.}$$

Usamos este mismo conteo en las tres versiones, aunque la beta 3 calcule la distancia con un producto de matrices, para que las tasas sean comparables. La velocidad es $\text{FLOP/s} = \text{FLOP}_{total}/T_{ej}$ [8].

3. Resultados y análisis

3.1. Tiempos de ejecución, cómputo y comunicación

La Figura 4 muestra T_{ej}, T_{comp} y T_{comm} para los cuatro tamaños, y la Tabla 4 los valores para n_0 y $16n$. El cómputo baja casi como $1/p$, y la comunicación crece poco con p: en $16n$ sube de 3,7 a 10,0 ms, y en n_0 se mantiene entre 0,42 y 0,53 ms hasta $p = 6$ y llega a 0,65 ms con $p = 8$, porque la domina el scatter, cuyo término de datos no depende de p. Para $4n$ y $16n$ la comunicación es entre 27 y 2700 veces menor que el cómputo (granularidad $G = T_{comp}/T_{comm}$ [5]), así que el problema es *compute bound* en todo el rango. Para $n/4$, en cambio, con $p = 8$ la comunicación ya es casi igual al cómputo ($G \approx 1{,}2$) y el tiempo total vuelve a subir. La precisión de cada tamaño es la misma para todo p (Figura 3).

p	T_{ej} (ms)	T_{comp} (ms)	T_{comm} (ms)	G	S	E	$\Delta T/\Delta p$	GFLOP/s	desbal.
n_0: $n_{tr} = 1437$, $n_{te} = 360$									
1	26,96 ± 0,99	26,26	0,52	50,9	0,99	0,99	–	3,68	1,00
2	12,61 ± 0,05	12,15	0,46	26,4	2,13	1,06	-14,35	7,87	1,01
4	5,76 ± 0,08	5,41	0,42	12,8	4,65	1,16	-3,42	17,23	1,02
6	3,83 ± 0,24	3,51	0,53	6,6	7,00	1,17	-0,97	25,92	1,05
8	3,74 ± 1,03	3,36	0,65	5,1	7,17	0,90	-0,05	26,56	1,22
$16n$: $n_{tr} = 22\,992$, $n_{te} = 5760$									
1	9832,40 ± 138,11	9859,61	3,66	2695,7	0,99	0,99	–	2,59	1,00
2	4522,13 ± 116,41	4518,00	5,41	834,7	2,16	1,08	-5310,27	5,62	1,01
4	2165,59 ± 28,59	2153,61	5,74	375,4	4,50	1,13	-1178,27	11,74	1,01
6	1355,78 ± 167,83	1357,21	7,21	188,3	7,20	1,20	-404,90	18,75	1,01
8	1055,70 ± 34,53	1024,92	9,99	102,6	9,24	1,16	-150,04	24,09	1,02

Tabla 4: Resultados por p: mediana de 4 repeticiones; ± es la mitad del rango entre la más rápida y la más lenta. $\Delta T/\Delta p$ en ms por proceso; desbalance = máximo entre promedio de $T_{comp_1,q}$.

La Figura 5 separa las tres colectivas. El scatter es la más cara porque el maestro envía todo el entrenamiento ($n_{tr}\, b$, casi constante en p); el broadcast crece como $\log p$, y el gather crece con p porque el maestro recibe $p\, n_{te}\, k$ candidatos. Las latencias se ajustan con $\log p$, como en MPI (Sección 2.4). Todos estos tiempos son del orden de 10^{-4} a 10^{-2} s, así que tienen más ruido relativo que el cómputo.

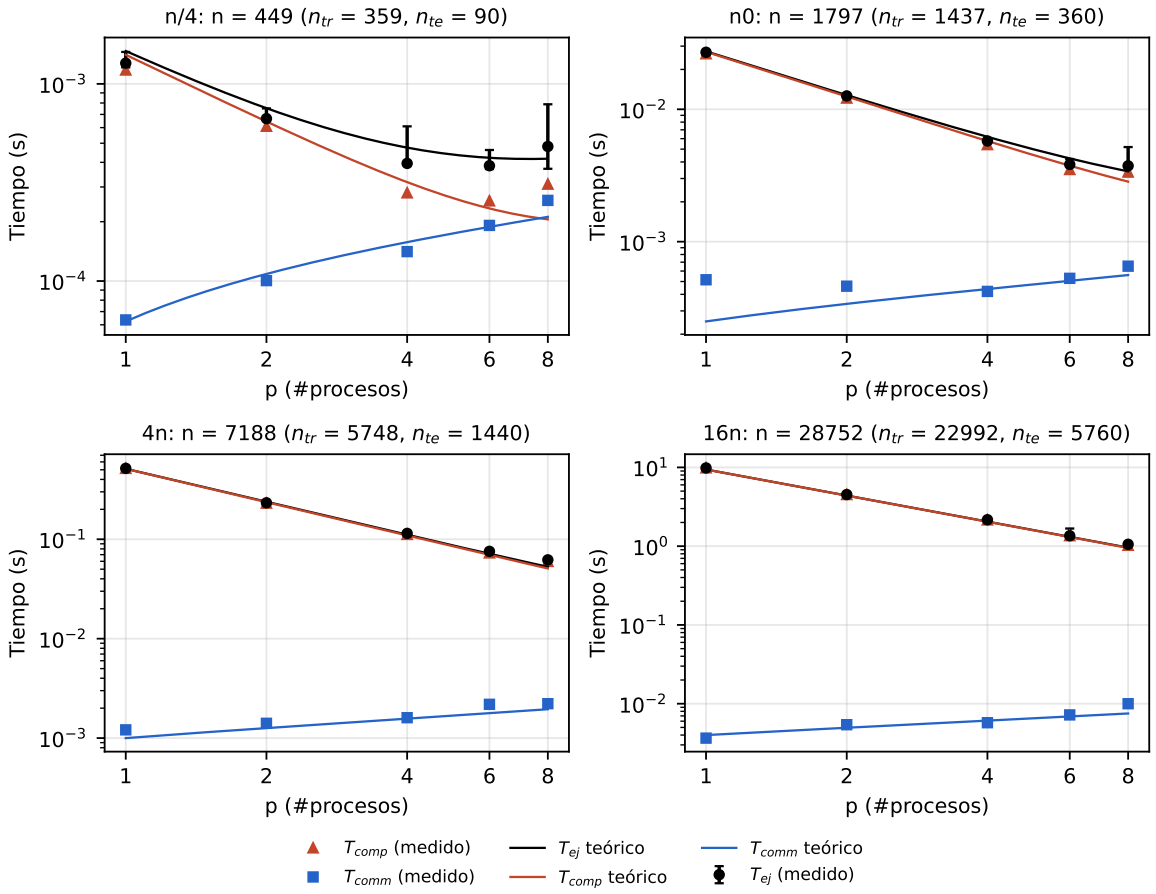


Figura 4: Tiempos de ejecución, cómputo y comunicación medidos (marcadores: mediana; barras: mínimo y máximo) y modelo teórico normalizado (líneas), para cada tamaño. Fuente: elaboración propia.

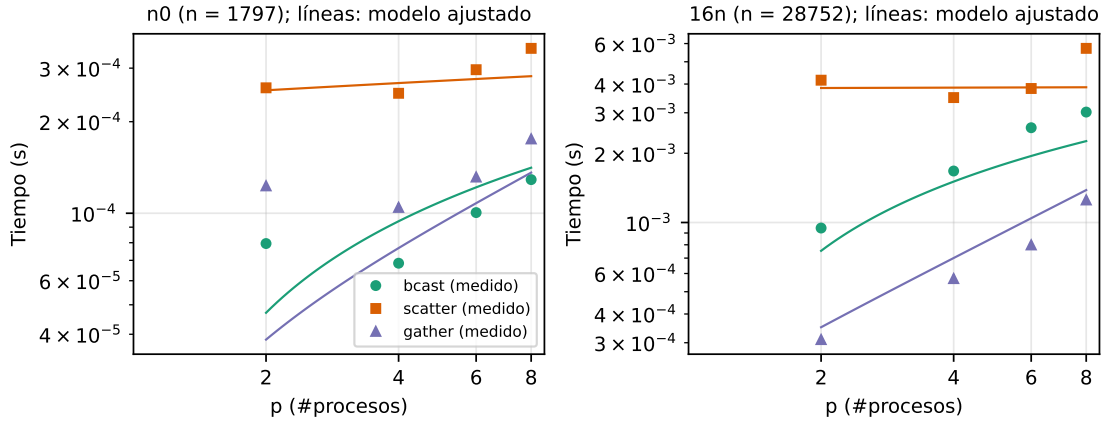


Figura 5: Tiempo de cada colectiva (modo B) y su modelo ajustado. Fuente: elaboración propia.

3.2. Comparación de las versiones beta

Con los datos originales, pasar del bucle de Python (beta 1) a NumPy (beta 2) reduce el tiempo 19 veces, y el producto de matrices (beta 3) lo reduce otras 2 veces (Figura 6). La beta 3 con un proceso tarda 27,0 ms frente a 1024 ms del código base. El orden del algoritmo es el mismo en las tres; lo que cambia es la constante de cada operación. En *speedup* relativo, la beta 1 (6,3 con $p = 8$) supera a la beta 2 (5,2) porque el cómputo local de la beta 2 se reparte peor con $p = 8$: su proceso más lento

tarda 1,2 veces el promedio (1,05 en la beta 1), y con cómputos de pocos ms cualquier interrupción pesa más. La comunicación pesa lo mismo en las dos (5,6 % del tiempo en el modo B). La beta 3 llega a 7,2 porque además su costo por par baja con $\log_2(n_{tr}/p)$ (Sección 3.5).

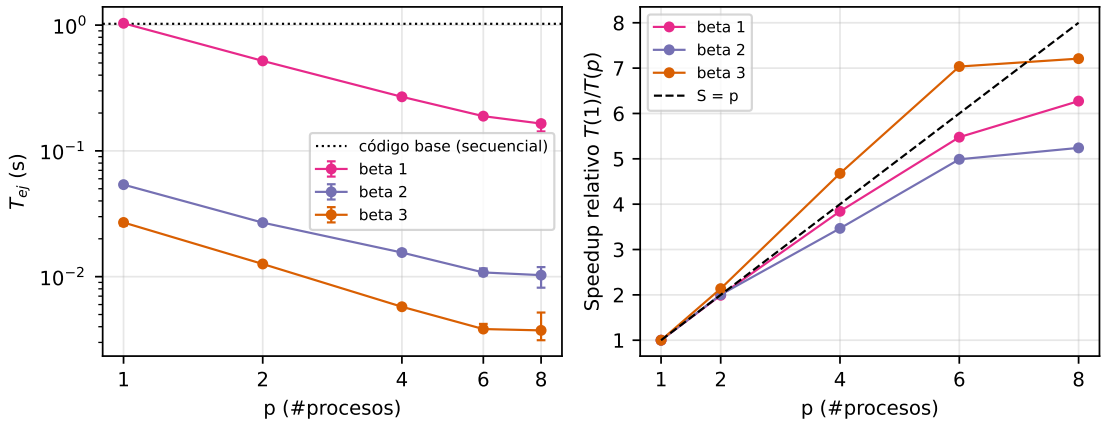


Figura 6: Versiones beta con $n = 1797$: tiempo de ejecución y *speedup* relativo $T(1)/T(p)$. Fuente: elaboración propia.

3.3. Normalización de la expresión teórica

Para comparar la expresión $O(\cdot)$ con los tiempos medidos, multiplicamos cada término por una constante física, de la misma forma que $T_{CPU} = n_{instr} \cdot CPI \cdot t_{cycle}$ y $T(m) = \alpha + m\beta$ [8]:

$$T_p^{teo}(n,p) = \underbrace{n_{te}\frac{n_{tr}}{p}\left(c_d + c_s \log_2 \frac{n_{tr}}{p}\right)}_{T_{comp_1}} + \underbrace{c_2\, n_{te}\, k\, p}_{T_{comp_2}} + \underbrace{\log_2 p\,(a_1 + n_{te} b_1)}_{\text{bcast}}$$
$$+ \underbrace{a_2 \log_2 p + n_{tr} b_2}_{\text{scatter}} + \underbrace{a_3 \log_2 p + p\, n_{te} k\, b_3}_{\text{gather}}$$

Cada constante la obtuvimos de su propia fase medida en el modo B (Tabla 5), con mínimos cuadrados ponderados por el error relativo: para T_{comp_1} usamos todos los (n,p) y para las colectivas solo $p \geq 2$ (con $p = 1$ no hay mensajes entre procesos); c_2 es la mediana de $T_{comp_2}/(n_{te}kp)$. En el ajuste del broadcast, a_1 salió negativo; como no tiene sentido físico, lo fijamos en 0 y volvimos a ajustar b_1. No usamos el tiempo total para ajustar, así que la comparación con T_{ej} sirve para validar el modelo, aunque no es del todo independiente: los modos A y B se miden en la misma ejecución.

Constante	Término	Valor	Se obtiene de
c_s	ordenamiento (paso 4)	4,64 ns por par y nivel	T_{comp_1} medido
c_d	distancias (paso 3)	3,79 ns por par	T_{comp_1} medido
c_2	mezcla y votación (paso 6)	41 ns por candidato	T_{comp_2} medido
a_1, b_1	broadcast	0 (ajuste libre < 0); 0,131 μs/dato (512 B)	T_{bcast} medido
a_2, b_2	scatter	14,3 μs por nivel; 0,167 μs/dato (512 B)	$T_{scatter}$ medido
a_3, b_3	gather	17,6 μs por nivel; 9,6 ns/candidato (24 B)	T_{gather} medido

Tabla 5: Constantes de la normalización (beta 3).

Las dos constantes del cómputo local quedan cerca de lo que mide la prueba aparte (`pruebas_extra.py`): $c_d = 3{,}8$ frente a 5,2 ns por par para las distancias, y $c_s = 4{,}6$ frente a 4,4 ns por par y nivel para el ordenamiento.

El modelo normalizado reproduce T_{ej} con $R^2 = 0{,}998$ y un error relativo medio de 8 % (5–6 % para n_0, $4n$ y $16n$; 14 % para $n/4$, donde los tiempos son de 1,3 ms o menos y el ruido pesa más). Dos resultados del ajuste son importantes:

Variante	α (μs)	β (ns/dato)	GB/s	Error máx.
búfer (`Send`/`Recv`)	0,52	0,425	18,8	68 %
objetos (`send`/`recv`, *pickle*)	5,28	1,975	4,1	119 %

Tabla 6: Ping-pong entre dos procesos, $T(m) = \alpha + m\,\beta$ con m datos *float64*: α es la mediana con $m \leq 100$ y β la pendiente con $m \geq 10^4$. El error máximo de la recta es grande porque el costo por dato cambia con el tamaño del mensaje (caché).

- **El ordenamiento domina el cómputo local.** En el PRAM supusimos que las distancias pesan más que el ordenamiento porque $d = 64 > \log_2 n_{tr}$. Eso vale para la beta 1, pero en la beta 3 las distancias salen de un producto de matrices: en una prueba aparte con un proceso (`pruebas_extra.py`), el paso 3 cuesta 5,2 ns por par y el paso 4 (ordenar) 46,4 ns por par, es decir 4,4 ns por nivel de $\log_2 n_{tr}$. El ordenamiento es cerca del 90 % del cómputo local, y por eso el costo por par crece con $\log_2(n_{tr}/p)$ (Figura 7). El orden del paso 3–4 es el de la Tabla 1, $O(n_{te}\,(n_{tr}/p)(d + \log(n_{tr}/p)))$; lo que cambió es cuál de los dos términos pesa.
- **Latencias y costo por dato.** Medimos el tiempo de un mensaje entre dos procesos con un ping-pong (`ping_pong.py`, Tabla 6). Con objetos de Python (*pickle*, como las colectivas en minúscula), $\alpha = 5{,}3$ μs y $\beta = 2{,}0$ ns por *float64*; con búferes de NumPy, $\alpha = 0{,}5$ μs y $\beta = 0{,}43$ ns. Los costos por dato del ajuste coinciden con la variante con objetos: b_1, b_2 y b_3 equivalen a 2,0, 2,6 y 3,2 ns por valor. Las latencias por nivel del scatter y del gather (14 y 18 μs) son mayores que α, porque cada colectiva arma y serializa los mensajes de todos los procesos. Las latencias quedan por encima del rango 10^{-7}–10^{-9} s que da el curso para α y β [5], y el costo por byte ($\approx$ 0,25–0,40 ns/B) por debajo, porque todos los procesos están en la misma computadora y los datos se copian por memoria.

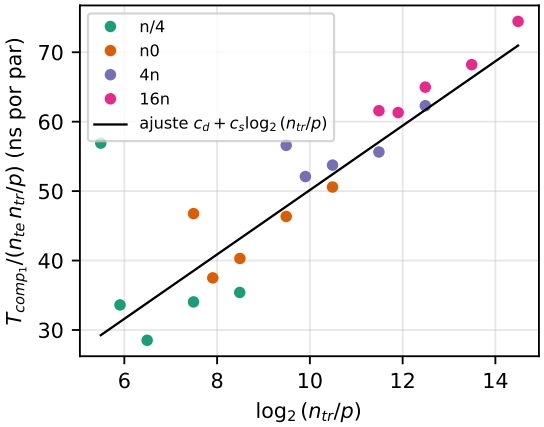


Figura 7: Costo del cómputo local por par prueba–entrenamiento en función de $\log_2(n_{tr}/p)$, para todos los (n, p), y el ajuste. Fuente: elaboración propia.

3.4. Cantidad óptima de procesos

Agrupando el modelo como $T(p) = A/p + B\,p + C\log_2 p + D$, con $A = n_{tr}\,n_{te}\,(c_d + c_s \log_2(n_{tr}/p))$, $B = c_2 n_{te} k + n_{te} k\,b_3$ y $C = a_1 + a_2 + a_3 + n_{te} b_1$, y tomando $\log_2(n_{tr}/p)$ aproximadamente constante cerca del óptimo, el número óptimo de procesos se obtiene de [8]:

$$T'(p) = -\frac{A}{p^2} + B + \frac{C}{p \ln 2} = 0 \;\Rightarrow\; p_{opt} = \frac{-C/\ln 2 + \sqrt{(C/\ln 2)^2 + 4AB}}{2B} \approx \sqrt{\frac{A}{B}}$$

La segunda derivada en p_{opt} es positiva ($p^2 T'' = 2Bp + C/\ln 2 > 0$), así que es un mínimo. Como $A \propto n^2$ y $B \propto n\,k$, p_{opt} crece aproximadamente como $\sqrt{n/k}$ (en la Tabla 7, algo más rápido, por

el factor log). Como la fórmula es aproximada, la Tabla 7 usa el mínimo numérico de T_p^{teo}. La tabla compara tres estimaciones: la de la expresión T_p de la Sección 2.4 sin constantes ($a = b = 1$ y $t_c = 1$, con t_c el tiempo de una distancia), la del modelo normalizado y la medida.

Tamaño	$\mathbf{n_{tr}}$	$\mathbf{n_{te}}$	$\mathbf{p_{opt}}$ (a=b=t_c=1)	$\mathbf{p_{opt}}$ **(normalizado)**	**Mínimo medido** ($p \leq 8$)
$n/4$	359	90	7,6	7,3	6
n_0	1 437	360	15,4	18,4	8
$4n$	5 748	1 440	30,8	40,1	8
$16n$	22 992	5 760	61,8	84,9	8

Tabla 7: Cantidad óptima de procesos.

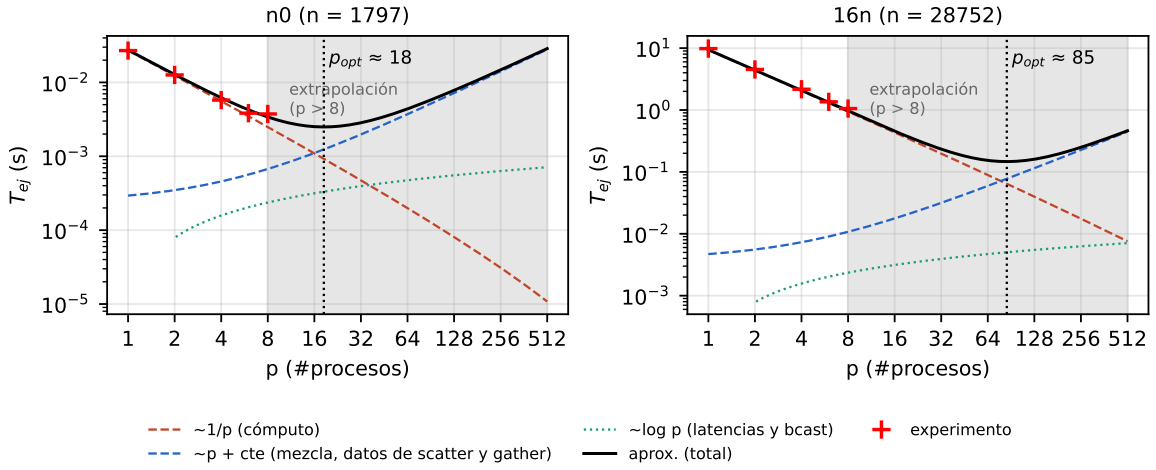


Figura 8: Modelo normalizado y sus términos (líneas) frente a las mediciones (cruces rojas) para n_0 y $16n$. La zona gris ($p > 8$) es extrapolación. Fuente: elaboración propia.

Respuesta. Para el problema del enunciado ($n = 1797$) estimamos $p_{opt} \approx 18$ procesos: hasta ahí el cómputo, que baja como $1/p$, pesa más que la mezcla del maestro y los datos del gather, que crecen como p (Figura 8). En nuestra computadora, con 8 núcleos de rendimiento, el mejor valor medido es $p = 8$, aunque pasar de 6 a 8 procesos solo ahorra 0,1 ms ($\Delta T/\Delta p = -0{,}05$ ms por proceso), así que en la práctica $p = 6$–8 es el punto razonable. Para $n/4$ el mínimo sí cae dentro del rango: el modelo da $p_{opt} \approx 7$ y el mínimo medido está en $p = 6$ (con $p = 4$ casi igual). Para $4n$ y $16n$ el modelo da $p_{opt} \approx 40$ y 85, fuera de los 8 núcleos, por lo que en esos casos conviene usar todos los núcleos disponibles.

3.5. Speedup y FLOP/s

La Figura 9 muestra el *speedup*, calculado con los tiempos T_{ej} de la Sección 3.1. Para $n/4$ el *speedup* llega a 3,1 con $p = 6$ y luego baja, por el *overhead*. Para n_0 llega a 7,2 con $p = 8$ y supera la recta $S = p$ con $p = 2$ a 6; para $4n$ y $16n$ la supera con todo $p \geq 2$ ($S = 9{,}2$ con $p = 8$ en $16n$). Un *speedup* superlineal es raro [8]; aquí lo explica el término de ordenamiento: cada proceso ordena listas de n_{tr}/p elementos, así que su costo por par baja con $\log_2(n_{tr}/p)$, mientras que T_s ordena listas de n_{tr}. El modelo normalizado predice este efecto ($T^{teo}(1)/T^{teo}(8) = 9{,}8$ para $16n$). Si T_s fuera un algoritmo secuencial que no ordena todo (por ejemplo, con selección de los k menores), el *speedup* ya no sería superlineal.

La velocidad en FLOP/s (Figura 9, derecha) va de 2,6–4,9 GFLOP/s con un proceso a 24–27 GFLOP/s con 8 procesos (salvo $n/4$, que llega a 16,1 GFLOP/s con $p = 6$ y baja a 12,9 con $p = 8$). Como el número de FLOP es el mismo en paralelo y en secuencial, $\text{FLOPs}(p)/\text{FLOPs}_{sec} = T_s/T_p = S$: la gráfica de FLOP/s es el *speedup* multiplicado por la velocidad secuencial, pero muestra además el valor absoluto. Para $n/4$ la curva teórica se aplana cerca de $p \approx 7$ (unos 15 GFLOP/s, sin bajar con $p = 8$), pero la velocidad medida cae un 20 % de $p = 6$ a 8: el modelo no capta esa caída.

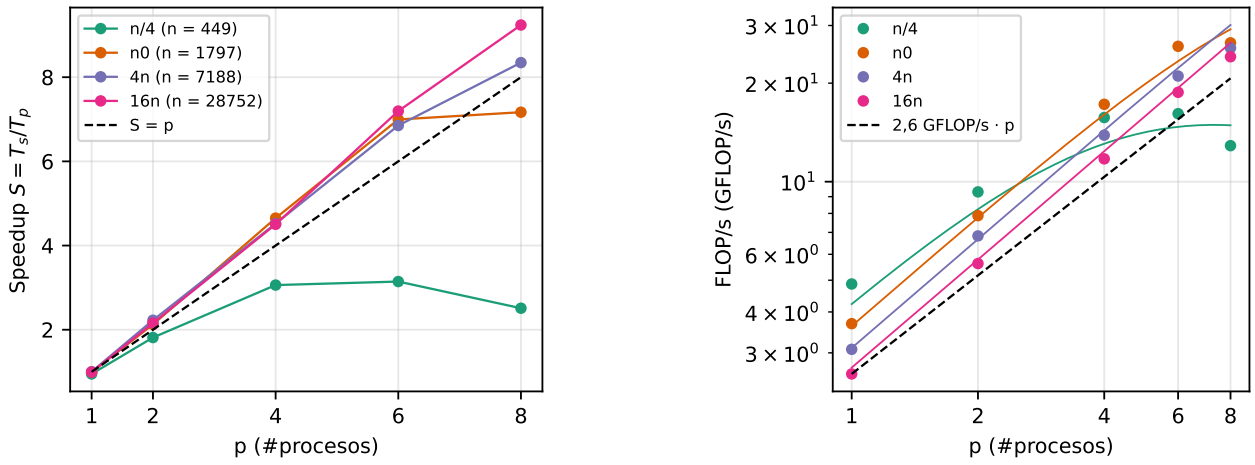


Figura 9: Izquierda: *speedup* $S = T_s/T_p$. Derecha: velocidad en FLOP/s (marcadores), la curva teórica $\text{FLOP}_{total}/T_p^{teo}$ (líneas) y la recta ideal. Fuente: elaboración propia.

3.6. Escalabilidad

Siguiendo el curso [8], la función de *overhead* es $T_0 = p\,T_p - W$, con $W = n_{tr}n_{te} = 0{,}16\,n^2$ y todo en función de un solo n:

$$T_0 = O\big(n\,k\,p^2(1+b) + a\,p\log p + b\,n\,p\,(1+\log p)\big), \qquad E = \frac{1}{1+T_0/W}.$$

Aquí contamos W en pares ($n_{tr}n_{te}$) y no en operaciones como en la Sección 2.3; como cada par cuesta $O(d) = O(1)$, las dos cuentas son $\Theta(n^2)$ y el análisis no cambia. W no incluye el ordenamiento (suponemos $d > \log_2 n_{tr}$). En la beta 3 cada proceso ordena listas de n_{tr}/p, así que $p\,T_{comp_1}$ es menor que T_s; por eso medimos $E > 1$. Si W incluyera el ordenamiento, T_0 ganaría el término negativo $-c_s\,n_{tr}n_{te}\log_2 p$ (cada proceso ordena listas p veces más cortas), que es el que da $E > 1$; cuando $T_0 < 0$, como en $4n$ y $16n$, $W = K\,T_0$ no tendría solución con $K > 0$. Por eso tomamos como W el costo del mejor secuencial, $\Theta(n^2)$ sin ordenar, y con ese W valen las condiciones de la Tabla 8. Para mantener E constante se necesita $W = K \cdot T_0$, con $K = E/(1-E)$ [8] (en mayúscula para no confundirla con los k vecinos), término a término (Tabla 8). Por la regla del máximo, la condición que manda es

$$n = \Theta(k\,p^2), \qquad \text{es decir} \qquad W = \Theta(k^2p^4)\,:$$

si p se duplica, n debe crecer 4 veces y el trabajo 16 veces. Con la división 80/20 fija, n_{tr} y n_{te} crecen juntos; si se pudiera cambiar esa proporción, bastaría $n_{te} = \Theta(p)$ y $n_{tr} = \Theta(k\,p^2)$, o sea $W = \Theta(k\,p^3)$, que es menos exigente.

Término de T_0	**Origen**	**Condición para $W = K\,T_0$**
$n\,k\,p^2(1+b)$	gather de candidatos y mezcla en el maestro	$n = \Theta(k\,p^2)$
$a\,p\log p$	latencias de broadcast, scatter y gather	$n = \Theta(\sqrt{p\log p})$
$b\,n\,p$	datos del scatter	$n = \Theta(p)$
$b\,n\,p\log p$	datos del broadcast	$n = \Theta(p\log p)$

Tabla 8: Condiciones de isoeficiencia, con un solo n ($W = \Theta(n^2)$).

Hicimos tres experimentos (Figuras 10 y 11, Tabla 9). El primero mira la escalabilidad fuerte, que es el punto de vista de Amdahl (n fijo), y los otros dos hacen crecer el problema con p, que es el punto de vista de Gustafson [5]:

- **E. Fuerte (n constante):** con n fijo el denominador de E crece como $k\,p^2 + p\log p + p$, así que el algoritmo **no es escalable** con n fijo. Se ve claramente en $n/4$ ($E = 0{,}31$ con $p = 8$) y en n_0

(E baja de 1,17 a 0,90 entre $p = 6$ y $p = 8$). Para $4n$ y $16n$, E se mantiene sobre 1 en $2 \le p \le 8$ (solo baja un poco entre $p = 6$ y 8) porque T_0/W es muy pequeño.

- **E. Débil (Gustafson, $n \propto \sqrt{p}$):** si n crece como $\sqrt{p}$, el trabajo por proceso queda constante ($W/p = 0{,}16\,n^2/p \approx 517\,320$ pares en los cuatro puntos). La eficiencia débil $E_w = T(1)/T(p)$ da 1,06 y 1,08 con $p = 2$ y 4, y baja a 0,83 con $p = 8$. Eso es lo que predice la teoría: con $n \propto \sqrt{p}$ el *overhead* relativo $k\,p^2/n$ crece como $p^{1,5}$, así que esta ley no mantiene E constante. Con $p \le 4$ la caída todavía no se ve porque las listas que ordena cada proceso se acortan (n_{tr}/p baja de 1437 a 719 y $\log_2(n_{tr}/p)$ de 10,5 a 9,5). Con $p = 8$ se suma el hardware compartido: en las 4 repeticiones de ese punto, algún proceso perdió entre 8 y 18 % de su tiempo de CPU frente a otros programas (Sección 3.7).
- **Isoeficiencia ($n \propto k\,p^2$):** de $n_0 = 1797$ a 115 008 imágenes, o sea 2 119 millones de pares con $p = 8$. La eficiencia se mantiene casi plana ($E = 0{,}99$, 1,11, 1,13 y 1,13 para $p = 1, 2, 4, 8$), que es justo lo que predice la condición $n = \Theta(k\,p^2)$. Con $p \le 8$ esto es consistente con la teoría pero no la prueba: el *overhead* todavía es mucho menor que el cómputo, y el E un poco mayor que 1 viene del ordenamiento (Sección 3.7).

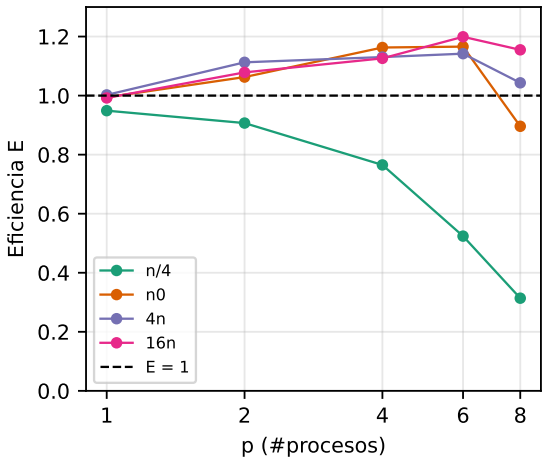


Figura 10: Eficiencia $E = T_s/(p\,T_p)$ con n fijo, para cada tamaño. Fuente: elaboración propia.

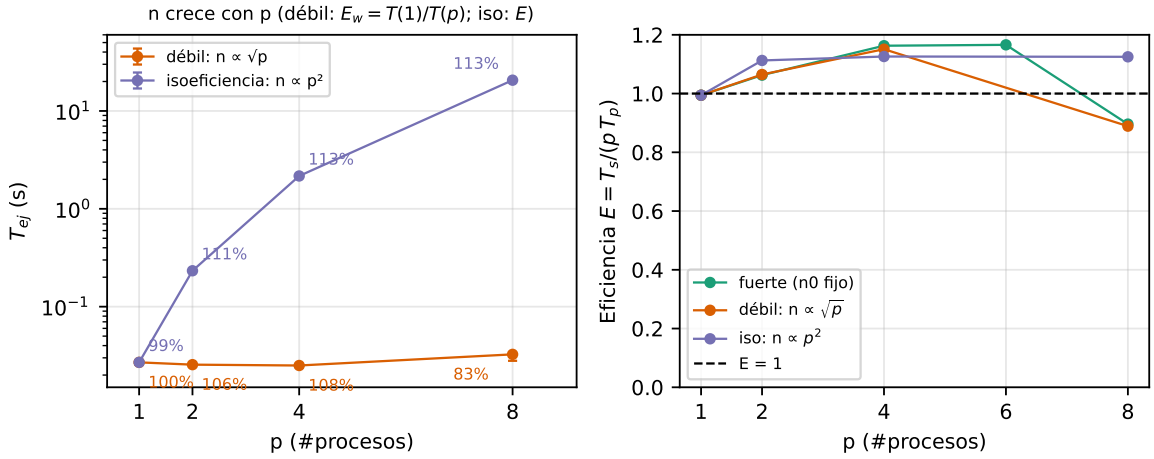


Figura 11: Izquierda: tiempo cuando n crece con p (en % la eficiencia débil E_w para la serie débil y E para la de isoeficiencia). Derecha: eficiencia de los tres experimentos. Fuente: elaboración propia.

Experimento	p	**n**	$\mathbf{n_{tr}}$	T_{ej} (ms)	T_s (ms)	E	E_w
débil ($n \propto \sqrt{p}$)	1	1 797	1 437	27,0	26,8	0,99	1,00
débil ($n \propto \sqrt{p}$)	2	2 541	2 032	25,5	54,4	1,06	1,06
débil ($n \propto \sqrt{p}$)	4	3 594	2 874	25,0	115,3	1,15	1,08
débil ($n \propto \sqrt{p}$)	8	5 082	4 064	32,5	231,5	0,89	0,83
iso ($n \propto p^2$)	1	1 797	1 437	27,0	26,8	0,99	–
iso ($n \propto p^2$)	2	7 188	5 748	232,8	518,0	1,11	–
iso ($n \propto p^2$)	4	28 752	22 992	2165,6	9754,9	1,13	–
iso ($n \propto p^2$)	8	115 008	91 968	20633,2	185718,4	1,13	–

Tabla 9: Escalabilidad débil e isoeficiencia, con un solo n ($n_{tr} = 0{,}8n$, $n_{te} = 0{,}2n$): mediana de 4 repeticiones (2 con 115 008 imágenes). La fila con $p = 1$ es la misma medición en las dos series.

3.7. Diferencias entre la teoría y el experimento

- **Constantes ocultas:** en la beta 3 el ordenamiento pesa más que las distancias, al revés de lo que supusimos con $d = 64$ en la entrega parcial. Esto da el *speedup* superlineal.
- **Hardware compartido:** en una prueba aparte (`pruebas_extra.py`) corrimos 8 copias independientes del mismo cálculo y cada una tardó 29,8 ms en promedio (31,4 ms la más lenta) frente a 26,6 ms sola (1,12 veces más; 1,02 con 4 copias). El mismo efecto se ve en la serie de control con n_{te} fijo y $n_{tr} \propto p$ (así $n_{tr}/p = 1437$ no cambia y el ordenamiento cuesta lo mismo): el cómputo local sube de 26,2 a 32,4 ms con $p = 8$, un 24 % más. Con $p = 8$ los 8 núcleos de rendimiento están ocupados, así que cualquier otro programa le quita CPU a algún proceso o lo hace correr en un núcleo de eficiencia; además, los procesos comparten memoria y frecuencia. Eso explica la caída de E_w con $p = 8$ en la serie débil, y que el modelo, que no incluye nada de esto, subestime el tiempo con $p = 8$ en $4n$ (52,8 frente a 62,1 ms) y en la isoeficiencia (17,6 frente a 20,6 s).
- **Serialización:** las colectivas en minúscula usan *pickle*. En el ping-pong, la latencia con objetos es 10 veces la de los búferes (5,3 frente a 0,5 μs) y el costo por dato 4,6 veces (2,0 frente a 0,43 ns).
- **Parte secuencial en el maestro:** la mezcla de los $p \cdot k$ candidatos crece con p y para $n/4$ ya es comparable al cómputo.
- **Desbalance:** como n_{tr} puede no ser múltiplo de p (por ejemplo, 1437), los bloques difieren en una fila; el desbalance medido es de 0–9 %, salvo n_0 con $p = 8$ (22 %) y $n/4$ con $p = 6$ y 8 (26 y 84 %), donde el cómputo por proceso dura menos de 3,5 ms y cualquier interrupción del sistema pesa más.

3.8. Mejoras

- **Seleccionar en vez de ordenar:** usar `np.argpartition` para los k menores ($O(n_{tr}/p)$ en lugar de $O(n_{tr}/p \log(n_{tr}/p))$). Como el ordenamiento domina, esto bajaría T_{comp_1} y quitaría el *speedup* superlineal.
- **Reducción en árbol en vez de gather (beta 3b, implementada):** mezclamos los candidatos de dos en dos con `comm.reduce` y un operador que deja los k menores. En teoría, el término $n\,k\,p$ pasa a $n\,k\log p$ y la condición de isoeficiencia baja de $n = \Theta(k\,p^2)$ a $n = \Theta(k\,p\log p)$ [5], [8]. Con $p = 8$, el tiempo de los pasos 5 a 7 baja un 33 % en $16n$ (de 6,9 a 4,6 ms), pero ahí esos pasos son menos del 1 % de T_{ej}; en n_0 y $4n$ casi no cambia (2 %). El tiempo total cambia a lo más un 4 % en n_0, $4n$ y $16n$, del orden del ruido; en $n/4$, donde una corrida dura alrededor de 1 ms, la 3b es hasta 9 % más lenta (Figura 12). Como anticipó el profesor, el ahorro es pequeño [10]: con $p \leq 8$ el gather todavía no es un cuello de botella.
- **Colectivas con búfer:** `Bcast`, `Scatterv` y `Gatherv` sobre arreglos de NumPy evitan *pickle*: en el ping-pong, la latencia baja de 5,3 a 0,5 μs y el costo por dato de 2,0 a 0,43 ns.
- **Evitar el scatter:** si cada proceso lee su propio bloque de entrenamiento, se elimina el término $b\,n_{tr}$, como en el algoritmo optimizado de la PD02 (Algorithm 6), que evita el scatter y el gather

[6].

- **Paralelismo híbrido:** MPI entre nodos e hilos dentro de cada nodo aumenta la granularidad, aunque no necesariamente es más eficiente [7].

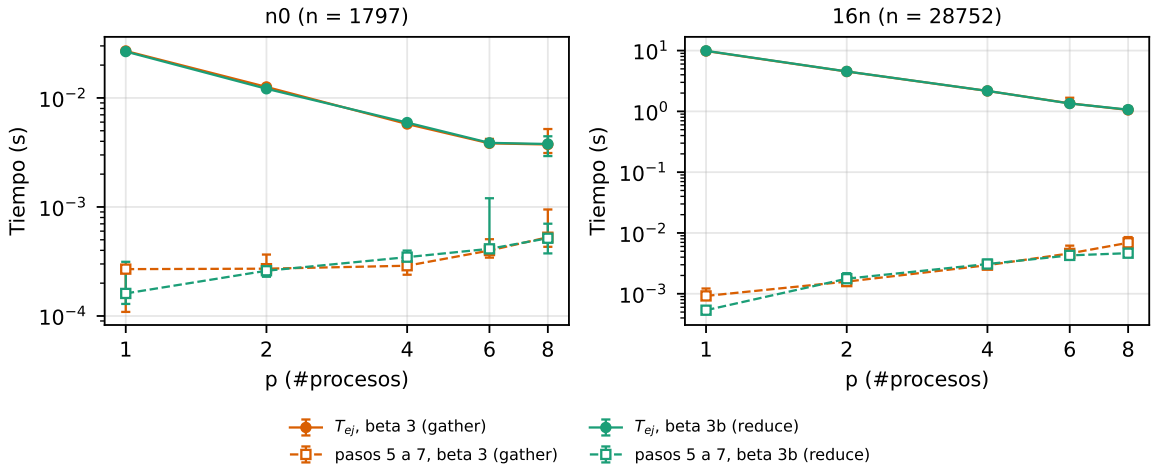


Figura 12: Beta 3 (gather) frente a beta 3b (reducción en árbol) con n_0 y $16n$: tiempo de ejecución y tiempo de los pasos 5 a 7. Fuente: elaboración propia.

3.9. Síntesis

El KNN en paralelo con bcast, scatter y gather mantiene exactamente las predicciones del código secuencial y es *compute bound* para los tamaños que medimos, salvo el más pequeño. La expresión teórica de la entrega parcial, normalizada con constantes medidas por fase, reproduce los tiempos con un error medio de 8 %. El número óptimo de procesos crece como $\sqrt{n}$ (unos 18 para $n = 1797$) y el algoritmo solo escala si n crece como $k\,p^2$. La reducción en árbol da las mismas predicciones, con un ahorro pequeño.

4. Bibliografía

[1] J. Fiestas, *Computación Paralela y Distribuida: Proyecto – Laboratorio*, 2026-II. Lima, Perú: UTEC, 2026.
Impacto: define el caso 5, el DAG (Figura 1) y lo que debe tener el informe final.

[2] `knn_digits_sec.py`, código secuencial del caso 5, adjunto al enunciado del proyecto [1]. [Online]. Available: `https://drive.google.com/file/d/1wTBSh7bwT1QVvceOpFMv5tcCWsg4-HeK/view` (accessed Sep. 18, 2026).
Impacto: es el código que paralelizamos y la referencia para validar las predicciones y la precisión.

[3] L. Dalcin, *MPI for Python (mpi4py) documentation* [Online]. Available: `https://mpi4py.readthedocs.io/en/stable/` (accessed Sep. 18, 2026).
Impacto: uso de `comm.bcast`, `comm.scatter`, `comm.gather`, `comm.Barrier` y `MPI.Wtime` en el código.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
Impacto: librería de los datos (`load_digits`) y del *split* del código base.

[5] J. Fiestas, diapositivas y apuntes del curso CS4052 (2026-II): U1.2 Foster y Flynn, U1.3 DAG, U2.1 Performance, U2.2 y U2.3 PRAM, U2.4 Particionamiento. Lima, Perú: UTEC, 2026.
Impacto: base teórica del PRAM, de S y E, del modelo $T_{comm} = f(p)(\alpha + x\beta)$, de la granularidad $G = T_{comp}/T_{comm}$ y de la mejora con reducción en árbol.

[6] J. Fiestas, *CS4052 – Prácticas Dirigidas 01 y 02 (soluciones)*. Lima, Perú: UTEC, 2026.
Impacto: modelo del pseudocódigo distribuido, de la forma $E = O(1/(1 + \dots))$ y de la mejora sin scatter.

[7] J. Fiestas, diapositivas del curso CS4052: U4.5 Paralelismo híbrido. Lima, Perú: UTEC, 2026.
Impacto: usar un proceso MPI por núcleo físico y plantear el paralelismo híbrido como mejora.

[8] J. Fiestas, diapositivas del curso CS4052: U6.1 Performance–Broadcast y U6.2 Performance y escalabilidad, y código `knn.cpp` de la PD04 (material del semestre 2026-I). Lima, Perú: UTEC, 2026.
Impacto: modelo $T(m) = \alpha + m\beta$ de las colectivas y reducción en árbol; conteo de FLOP y FLOP/s; versión secuencial considerada para T_s y *speedup* superlineal; función de *overhead* T_0, isoeficiencia y regla del máximo; cálculo de p_{opt} con $T'(p) = 0$; datos sintéticos para escalar el problema.

[9] Anthropic, *Claude Opus 5* y *Claude Opus 5.5* (asistentes de inteligencia artificial, usados con Claude Code), setiembre a noviembre de 2026.
Uso: resumió el material del curso; escribió el código de las tres versiones beta, de la mejora con reducción y de los scripts de experimentos, ping-pong, validación y análisis; ejecutó las mediciones; y redactó el borrador de este informe y de la presentación.
Impacto y validación: la mayor parte del código y del texto se hizo con esta herramienta. Validamos el código comparando las predicciones con las del código base del profesor para $p = 1$ a 8 (Sección 2.6) y revisamos las fórmulas y el análisis con el material del curso [5]–[8]. Las cifras del análisis salen de los datos y scripts del .zip adjunto. El grupo es responsable del contenido presentado.

[10] J. Fiestas, respuesta por correo electrónico a una consulta del grupo sobre el costo de `scatter` y `gather` y el acceso al clúster Khipu, 6 de noviembre de 2026 (comunicación personal).
Impacto: aclara que $O(p)$ es la aproximación no optimizada y que MPI usa $O(\log p)$, con la expresión de la U2.4 (Tabla 1); acepta la reducción en árbol como mejora (Sección 3.8); y justifica medir en una computadora personal porque Khipu no estaba disponible (Sección 2.7).